

4.3 Abstract Types

Types such as **complex** and **Vector** are called *concrete types* because their representation is part of their definition. In that, they resemble built-in types. In contrast, an *abstract type* is a type that completely insulates a user from implementation details. To do that, we decouple the interface from the representation and give up genuine local variables. Since we don't know anything about the representation of an abstract type (not even its size), we must allocate objects on the free store (§4.2.2) and access them through references or pointers (§1.7, §13.2.1).

First, we define the interface of a class **Container**, which we will design as a more abstract version of our **Vector**:

```
class Container {
public:
    virtual double& operator[](int) = 0;           // pure virtual function
    virtual int size() const = 0;                 // const member function (§4.2.1)
    virtual ~Container() {}                      // destructor (§4.2.2)
};
```

This class is a pure interface to specific containers defined later. The word **virtual** means “may be redefined later in a class derived from this one.” Unsurprisingly, a function declared **virtual** is called a *virtual function*. A class derived from **Container** provides an implementation for the **Container** interface. The curious **=0** syntax says the function is *pure virtual*; that is, some class derived from **Container** *must* define the function. Thus, it is not possible to define an object that is just a **Container**. For example:

```
Container c;                               // error: there can be no objects of an abstract class
Container* p = new Vector_container(10);    // OK: Container is an interface
```

A **Container** can only serve as the interface to a class that implements its **operator[]()** and **size()** functions. A class with a pure virtual function is called an *abstract class*.

This **Container** can be used like this:

```
void use(Container& c)
{
    const int sz = c.size();

    for (int i=0; i!=sz; ++i)
        cout << c[i] << '\n';
}
```

Note how **use()** uses the **Container** interface in complete ignorance of implementation details. It uses **size()** and **[]** without any idea of exactly which type provides their implementation. A class that provides the interface to a variety of other classes is often called a *polymorphic type*.

As is common for abstract classes, **Container** does not have a constructor. After all, it does not have any data to initialize. On the other hand, **Container** does have a destructor and that destructor is **virtual**, so that classes derived from **Container** can provide implementations. Again, that is common for abstract classes because they tend to be manipulated through references or pointers, and someone destroying a **Container** through a pointer has no idea what resources are owned by its implementation; see also §4.5.

The abstract class **Container** defines only an interface and no implementation. For **Container** to be useful, we have to implement a container that implements the functions required by its interface. For that, we could use the concrete class **Vector**:

```
class Vector_container : public Container { // Vector_container implements Container
public:
    Vector_container(int s) : v(s) { } // Vector of s elements
    ~Vector_container() {}

    double& operator[](int i) override { return v[i]; }
    int size() const override { return v.size(); }
private:
    Vector v;
};
```

The **:public** can be read as “is derived from” or “is a subtype of.” Class **Vector_container** is said to be *derived* from class **Container**, and class **Container** is said to be a *base* of class **Vector_container**. An alternative terminology calls **Vector_container** and **Container** *subclass* and *superclass*, respectively. The derived class is said to inherit members from its base class, so the use of base and derived classes is commonly referred to as *inheritance*.

The members **operator[]()** and **size()** are said to *override* the corresponding members in the base class **Container**. I used the explicit **override** to make clear what’s intended. The use of **override** is optional, but being explicit allows the compiler to catch mistakes, such as misspellings of function names or slight differences between the type of a **virtual** function and its intended overrider. The explicit use of **override** is particularly useful in larger class hierarchies where it can otherwise be hard to know what is supposed to override what.

The destructor (**~Vector_container()**) overrides the base class destructor (**~Container()**). Note that the member destructor (**~Vector()**) is implicitly invoked by its class’s destructor (**~Vector_container()**).

For a function like **use(Container&)** to use a **Container** in complete ignorance of implementation details, some other function will have to make an object on which it can operate. For example:

```
void g()
{
    Vector_container vc(10); // Vector of ten elements
    // ... fill vc ...
    use(vc);
}
```

Since **use()** doesn’t know about **Vector_containers** but only knows the **Container** interface, it will work just as well for a different implementation of a **Container**. For example:

```
class List_container : public Container { // List_container implements Container
public:
    List_container() { } // empty List
    List_container(initializer_list<double> il) : l{il} { }
    ~List_container() {}
```

```

    double& operator[](int i) override;
    int size() const override { return ld.size(); }
private:
    std::list<double> ld;    // (standard-library) list of doubles (§11.3)
};

double& List_container::operator[](int i)
{
    for (auto& x : ld) {
        if (i==0)
            return x;
        --i;
    }
    throw out_of_range{"List container"};
}

```

Here, the representation is a standard-library `list<double>`. Usually, I would not implement a container with a subscript operation using a `list`, because performance of `list` subscripting is atrocious compared to `vector` subscripting. However, here I just wanted to show an implementation that is radically different from the usual one.

A function can create a `List_container` and have `use()` use it:

```

void h()
{
    List_container lc = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    use(lc);
}

```

The point is that `use(Container&)` has no idea if its argument is a `Vector_container`, a `List_container`, or some other kind of container; it doesn't need to know. It can use any kind of `Container`. It knows only the interface defined by `Container`. Consequently, `use(Container&)` needn't be recompiled if the implementation of `List_container` changes or a brand-new class derived from `Container` is used.

The flip side of this flexibility is that objects must be manipulated through pointers or references (§5.2, §13.2.1).

4.4 Virtual Functions

Consider again the use of `Container`:

```

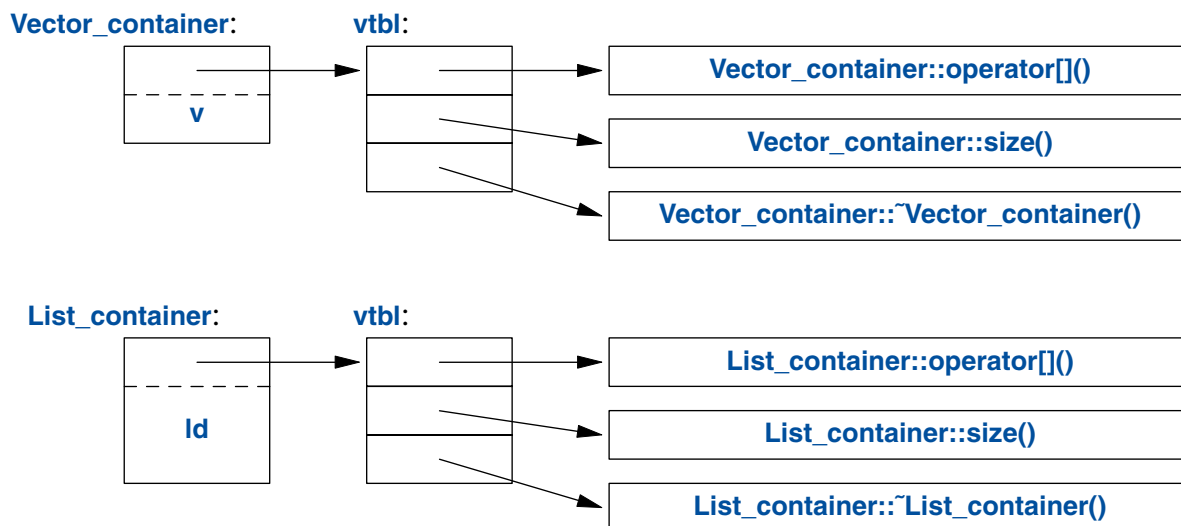
void use(Container& c)
{
    const int sz = c.size();

    for (int i=0; i!=sz; ++i)
        cout << c[i] << '\n';
}

```

How is the call `c[i]` in `use()` resolved to the right `operator[]()`? When `h()` calls `use()`, `List_container`'s `operator[]()` must be called. When `g()` calls `use()`, `Vector_container`'s `operator[]()` must be called. To

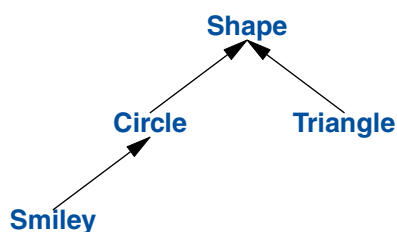
achieve this resolution, a **Container** object must contain information to allow it to select the right function to call at run time. The usual implementation technique is for the compiler to convert the name of a virtual function into an index into a table of pointers to functions. That table is usually called the *virtual function table* or simply the **vtbl**. Each class with virtual functions has its own **vtbl** identifying its virtual functions. This can be represented graphically like this:



The functions in the **vtbl** allow the object to be used correctly even when the size of the object and the layout of its data are unknown to the caller. The implementation of the caller needs only to know the location of the pointer to the **vtbl** in a **Container** and the index used for each virtual function. This virtual call mechanism can be made almost as efficient as the “normal function call” mechanism (within 25%). Its space overhead is one pointer in each object of a class with virtual functions plus one **vtbl** for each such class.

4.5 Class Hierarchies

The **Container** example is a very simple example of a class hierarchy. A *class hierarchy* is a set of classes ordered in a lattice created by derivation (e.g., **: public**). We use class hierarchies to represent concepts that have hierarchical relationships, such as “A fire engine is a kind of a truck which is a kind of a vehicle” and “A smiley face is a kind of a circle which is a kind of a shape.” Huge hierarchies, with hundreds of classes, that are both deep and wide are common. As a semi-realistic classic example, let’s consider shapes on a screen:



The arrows represent inheritance relationships. For example, class **Circle** is derived from class